GUAVA × gaussian-vrm 統合技術仕様書（完全版・詳細実装マニュアル）

1. 概要

本ドキュメントは、単一画像からサブ秒でアセットを生成する**GUAVA**論文（Section 3.2, 4.1）に基づいた画像エンコーダの仕様、および現状の実装における不整合の特定と修正指針、さらにスマホ駆動に最適化された**gaussian-vrm**との統合プロセスを定義するものである[1111111]。

既存の「Three.jsで最終画を作成する」という設計では、実装フェーズで顕在化した課題（WebGL・Three.js・Neural Refiner・Canvas依存）により破綻することが明らかになった[2222]。本仕様は「論文再現」ではなく、スマートフォン上で“人と自然に会話できる実写品質AIコンシェルジュ”を成立させることを最終目的とした現実的進化系である[3333]。

---

2. GUAVA 論文要約：次世代アバター生成の基盤技術

2.1 概要と目的

GUAVA (Generalizable Upper Body 3D Gaussian Avatar)は、単一画像入力・サブ秒推論という制約下で、顔・手を含む表情豊かな上半身アバターを生成する初の汎用フレームワークである[444444]。

- 従来手法の課題: マルチビュー必須、個人ごとの長時間最適化、サーバー依存[555]。
- **GUAVA**の革新点: 単一画像から約0.1秒（sub-second）の高速・フィードフォワード推論で、即座に高品質なアセットを生成可能。未知の人物に対しても事前学習なしで汎用性を発揮する[66666666666666]。

2.2 核心技術：**EHM (Expressive Human Model)**

従来のSMPL-Xモデルが弱かった「顔の微細な表情（Expression）」の表現力を大幅に強化するために導入された[7777]。

- 補完内容: 顔筋肉レベルの表情表現、口唇・顎・頬の非線形変形[8]。
- 効果: 生成される初期状態の質が飛躍的に向上し、Neural Refinerの学習負荷を軽減。高精度なトラッキングを実現する[9999]。

2.3 パフォーマンス特性

- アセット生成速度: ~0.1秒[10101010]。

- レンダリング速度: 50FPS以上（GPU環境）[11111111]。
- 制約: 論文の前提はCUDA / ネイティブ環境であり、WebGL実装とは前提が異なる[12]。

---

3. 画像エンコーダ（DINOv2）と特徴量サンプリングの標準仕様

本モジュールは、入力画像から3Dガウシアンの各頂点に紐付く外観特徴量 (Appearance Feature)を抽出・投影する役割を担う[13]。

3.1 入力および基幹モデル仕様

- 入力解像度: 518×518 ピクセル（DINOv2推奨サイズ）[14]。
- 基幹エンコーダ: DINOv2-base[15]。
- パッチサイズ: 14×14[16]。
- 出力パッチ数: $(518/14) \times (518/14) = 37 \times 37$ パッチ[17]。
- 特徴次元数: 768次元[18]。

3.2 特徴量生成フロー

論文の定式化に基づき、外観特徴量 F_a は以下の工程で生成される[19]。

1. **DINOv2** 特徴抽出: $37 \times 37 \times 768$ 次元[20]。
2. **Convolutional Upsampling**: パッチ特徴をソース画像の解像度 (518×518) までアップサンプリングする[21]。
3. 次元圧縮: 最終的な特徴次元を 32 (UV Gaussians用) および 128 (Template Gaussians用) へ変換する[22]。

3.3 投影およびサンプリング (Equation 2)

各頂点 v^i における特徴量 f_p^i は、以下のサンプリング関数 S によって算出される[23]。

$$f_p^i = S(F_a, P(v^i, RT_s))$$

- RT_s: ソース画像のビュー行列[24]。
- P: 投影関数 (3D空間から2Dスクリーン座標への変換)[25]。
- F_a: 外観特徴マップ[26]。

---

4. 現状分析と課題（Root Cause Analysis）

現在、アバター画像が表示されない根本原因は、「特徴マップの解像度」と「投影計算の座標系」の致命的な不整合にある[27]。

4.1 仕様比較表[28]

項目	論文/推奨仕様	現状の実装	判定
DINOv2入力	518×518	518×518	OK
Feature Map解像度	**518×518**	**256×256 (固定)**	**NG**
投影計算解像度	**518×518**	**2816×1536 (JSON)**	**NG**
座標系の一致	一致している	完全に不一致	NG

4.2 障害のメカニズム

1. 座標の逸脱: source_camera.json の高解像度 (2816×1536) 基準で投影計算を行っている[29]。
2. サンプリング失敗: 特徴マップ (image-encoder.ts) が 256×256 で生成されているため、投影座標がマップ範囲外となり、全ての頂点で無効な値（0または真っ黒）がサンプリングされる[30]。

---

5. 修正実施指針：解像度の統一

論文の「Appearance feature map matching source image resolution」という記述を **518×518** と定義し、システム全体の解像度をこの値に統一する[31]。

5.1 ONNXエンコーダの再定義 (Python)

DINOv2の出力を 256 ではなく **518** にアップサンプリングするよう変換ロジックを修正する[32]。

Python
```
# 修正後: 37x37 -> 518x518 upsampling (論文準拠)
output_518 = F.interpolate(
```

```
    output_37,
    size=(518, 518),
    mode='bilinear',
    align_corners=False
)
```

33

5.2 フロントエンド実装の修正 (TypeScript)

gvrm.ts におけるカメラパラメータを、特徴マップの解像度 (518×518) に強制的に一致させる[34]。

TypeScript

```
// gvrm.ts - extractFeaturesメソッド
const camera = {
    viewMatrix: this.buildViewMatrix(),
    projMatrix: this.buildProjectionMatrix(),
    screenWidth: 518,  // 518へ固定
    screenHeight: 518 // 518へ固定
};
```

35

---

6. スマホ実装におけるレンダリング再設計

6.1 レンダラー選定：なぜ gaussian-vrm なのか

動的ガウシアンの多くは変形（Deformation）にニューラルネットワークの推論を必要とし、スマホ単体では遅延・発熱が発生する 36。また、標準的な点群レンダラーには「ボーン（骨格）」の概念がなく、人型としての制御が困難である 37。

naruya/gaussian-vrm の採用理由は以下の通り 38：

- **LBS (Linear Blend Skinning)**による高速演算: スマホのGPUが得意とする行列演算でガウス粒子を変形させるため、完全なローカル・リアルタイム駆動が可能[39393939]。
- **VRM**規格準拠: ボーン構造を持ち、既存の表情・口パク・視線制御資産と接続可能。開発効率が劇的に向上する[40404040]。

6.2 WebGL と Three.js の役割再定義

本仕様では、既存の「Three.js に見た目を任せる」設計を捨て、役割を以下のように再定義する[41]。

- **WebGL =** 特徴量生成エンジン[42][42][42][42]
  - Gaussian splat の可視性、オクルージョン、深度関係の計算[43]。
  - チャンネル分割された **coarse feature map** の生成[44]。
  - 見た目は作らない[45]。
- **Three.js =** 構造管理・インターフェース[46][46][46][46]
  - 主役ではないが、ボーン駆動、カメラ、シーン管理を担う[47]。
  - 24属性データ構造の維持[48][48][48][48]。

---

7. 属性拡張設計：17属性から24属性への昇華

GUAVAの生成能力と gaussian-vrm の駆動能力を統合するため、属性のギャップを埋める必要がある[49]。

- **GUAVA出力 (17属性)**: 位置(3)、法線(3)、基本色(3)、不透明度(1)、スケール(3)、回転(4)。これは「見た目（静止点群）」を定義する最小単位である[50][50][50][50]。
- **gaussian-vrm要求 (24属性)**: 上記に加え、ボーンインデックスとスキンウェイト（計7属性）を付加した24プロパティ定義。「運動学的意味（身体部位）」を持つ存在となる[51][51][51][51][51][51][51][51][51][51][51][51][51][51][51][51][51][51][51]。

7.1 拡張フロー

1. **GUAVA (17ch PLY)** 生成[52]。
2. **WebGL coarse feature map** 生成[53]。
3. **Neural Refiner**: 粗い特徴マップを補完・高精細化し、構造的情報の潜在的な土台を形成する。本実装では「人間らしさの最終決定者」と定義する[54][54][54][

CPUコピーコスト、色空間破壊、Safari不安定を回避するため。実写アバターにおいては致命的である[57]。

8.2 レンダリング構成

1. **WebGL**: GS coarse pass を実行[58]。
2. **Neural Refiner** 推論[59]。
3. 表示: Refiner出力を **WebGL texture** として直接表示。2D Canvas は一切不使用とする[60]。
4. **Shader**制御: gamma / sRGB は shader 内ですべて制御する[61]。

---

9. 結論と期待される効果

本修正およびアーキテクチャ統合により、投影座標系と特徴マップのピクセル配置が1:1で一致し、正しい外観情報を取得可能となる[62]。

検証指標:

- image-encoder.onnx の出力サイズが $(1, 128, 518, 518)$ であること[63]。
- サーバー不要、スマホ完結で、高解像・高表情な「完全スタンドアロン型AIコンシェルジュ」が安定動作すること[64646464]。